PROJECT REFERENCE GUIDE

AgroSense

Smart irrigation monitoring and control platform

Current implementation guide · Version 0.1.0 · 7 September 2026

Purpose: This document is the single project reference for the current AgroSense product scope, architecture, setup, API contracts, device integration, and implementation status.

Start here

Run the backend and frontend in separate terminals. The backend can start without PostgreSQL or MQTT; it uses seeded in-memory telemetry until those services are configured.

Terminal 1 · backend

```
cd backend
cp .env.example .env
npm install
npm run dev
```

Terminal 2 · frontend

```
cd frontend
cp .env.example .env
npm install
npm run dev
```

Open the dashboard at http://localhost:5173. The Express API runs at http://localhost:4000.

Production build: Use npm run build in both backend and frontend directories. Start the compiled backend with npm start.

Project at a glance

Area	Current choice
Product	IoT-enabled smart irrigation dashboard for farm monitoring and control
Frontend	React, TypeScript, Vite, local shadcn-style UI primitives, Lucide icons
Backend	Node.js, Express, TypeScript, Socket.IO, MQTT.js, Zod
Persistence	PostgreSQL through pg; in-memory fallback for development
Device	ESP32 sensor/controller sending telemetry over Wi-Fi
Telemetry protocol	MQTT topic agrosense/{device_id}/telemetry
Live UI protocol	Socket.IO events telemetry:update and dashboard:summary
Current status	Dashboard shell, live summary, validation, MQTT gateway, and PostgreSQL write-through are implemented

Requirements from the supplied structure document

The attached structure document is treated as the product/design reference. Its page descriptions are requirements for the AgroSense application, not additional instructions that override the user's request.

Module	Scope
Login	User login, password visibility, remember-me option, forgot-password flow, and optional Google login treatment.
Dashboard	Temperature, humidity, water level, rain status, flow rate, all-zone soil moisture, pump status, water usage, and system status.
Live Sensor Data	Real-time readings, zone-wise moisture, live moisture trends, current sensor status, tank status, and rain status.
Zones	Independent monitoring and management of Zone 1, Zone 2, and Zone 3, including crops, pump state, trends, comparison, and zone irrigation details.
System Control	Automatic/manual modes, zone pump control, moisture thresholds, schedules, flow monitoring, and water-tank status.
Irrigation History	Past events with date/time, zone, moisture before/after, duration, water used, pump status, filters, and charts.
Plant Analysis	Plant health, growth stage, recommended moisture, environmental conditions, plant response, water fingerprint, and recommendations.
Alerts	Low moisture, high temperature, low water, rainfall, pump, offline, system alerts, history, and notification settings.
Settings	Device ID, Wi-Fi settings, thresholds, notifications, system preferences, data management, and save settings.

Architecture

The ESP32 communicates over Wi-Fi. MQTT is the recommended device-to-backend transport because it is lightweight, reconnectable, and suitable for continuous sensor messages.

```
ESP32 sensors / pumps
        | Wi-Fi + MQTT
        v
Express backend
  |-- MQTT ingestion and validation
  |-- PostgreSQL write-through
  |-- Alert and irrigation services
  |-- Socket.IO live updates
        v
React dashboard
```

Repository structure

```
backend/
  src/config              Environment parsing
  src/middleware          Express error handling
  src/routes              Versioned API routes
  src/services            Telemetry store and MQTT service
  src/types               Shared telemetry types
  src/validation          Zod schemas
  prisma/schema.prisma    Reference application data model
frontend/
  src/components/ui       Local shadcn-style primitives
  src/App.tsx             Dashboard and navigation shell
  src/api.ts              REST client
  src/styles.css          AgroSense visual system and layout
  components.json         shadcn component configuration
```

Frontend implementation

The frontend currently provides the AgroSense dashboard shell and uses local shadcn-style components so the UI is editable and consistent within the repository.

- Button: navigation, panel actions, and action variants
- Card: metric cards, zone cards, and dashboard panels
- Badge: device, connection, and pump status
- Avatar: admin user chip
- Progress: zone soil-moisture levels
- Separator: topbar structure
- Lucide icons: consistent navigation and metric icon alignment

Implemented dashboard content includes connection status, five system metrics, soil moisture by zone, water-usage visualization, moisture trend, pump donut, system status, and a branded sustainability banner.

Environment configuration

Variable	Default / example	Purpose
PORT	4000	Express API port
CLIENT_ORIGIN	http://localhost:5173	Allowed frontend origin
DATABASE_URL	empty	PostgreSQL connection string; empty enables in-memory fallback
MQTT_ENABLED	false	Enable ESP32 MQTT ingestion
MQTT_URL	mqtt://localhost:1883	MQTT broker address
MQTT_USERNAME / MQTT_PASSWORD	empty	Optional broker credentials
MQTT_TOPIC_PREFIX	agrosense	Topic namespace prefix
VITE_API_URL	http://localhost:4000/api/v1	Frontend REST base URL
VITE_SOCKET_URL	http://localhost:4000	Frontend Socket.IO origin

Telemetry contract

The backend accepts a payload with a device ID, system readings, and one or more zone readings. The timestamp is optional; the backend assigns the current time when omitted.

```
{
  "device_id": "AGROSENSE01",
  "timestamp": "2026-09-07T10:30:00Z",
  "system": {
    "temperature": 29.5,
    "humidity": 64,
    "water_level": 78,
    "rain": false,
    "flow_rate": 1.8
  },
  "zones": [
    { "zone_id": 1, "soil_moisture": 28, "pump_status": true },
    { "zone_id": 2, "soil_moisture": 52, "pump_status": false },
    { "zone_id": 3, "soil_moisture": 21, "pump_status": true }
  ]
}
```

Express API

Method	Endpoint	Purpose
GET	/api/v1/health	Service health check
GET	/api/v1/dashboard/summary	Latest dashboard summary; optional device_id query parameter
GET	/api/v1/devices/:deviceId/latest	Latest raw telemetry for a device
POST	/api/v1/devices/:deviceId/telemetry	Validated HTTP telemetry simulator/fallback endpoint

Example telemetry test:

```
curl -X POST http://localhost:4000/api/v1/devices/AGROSENSE01/telemetry \
  -H 'content-type: application/json' \
  --data @telemetry.json
```

ESP32 and MQTT integration

Set MQTT_ENABLED=true in backend/.env when the broker is available. The backend subscribes to:

```
agrosense/{device_id}/telemetry
```

The ESP32 should publish validated JSON telemetry periodically and reconnect automatically after Wi-Fi loss. Future control commands should use separate command and acknowledgement topics:

```
agrosense/{device_id}/commands
agrosense/{device_id}/commands/ack
```

Safety requirement: The ESP32 should enforce local safety rules for low tank level, missing flow, maximum pump duration, and conflicting pump commands. It must not depend entirely on cloud connectivity for physical safety.

Current implementation status

Area	Status	Notes
Express backend	Implemented	TypeScript, validation, health, dashboard summary, telemetry endpoint
MQTT gateway	Implemented	Disabled by default; validates incoming ESP32 payloads and broadcasts live updates
PostgreSQL write-through	Implemented	Creates telemetry tables when DATABASE_URL is configured; falls back safely otherwise
React dashboard	Implemented	Dashboard shell, live connection state, cards, charts, zones, status panels
Authentication	Planned	Login UI is in the product scope; protected auth routes are not yet implemented
Pump control	Planned	Safety-oriented API and ESP32 command acknowledgement are next
History, alerts, plant analysis, settings	Planned	Navigation placeholders exist; domain modules remain to be built
Real hardware validation	Pending	Use an MQTT simulator before connecting physical pumps

Recommended build sequence

1. Confirm local setup with the seeded dashboard and HTTP telemetry endpoint.
2. Run an MQTT broker and replace HTTP simulator messages with ESP32 telemetry.
3. Connect PostgreSQL and verify telemetry hydration after backend restart.
4. Implement authentication and protected routes.
5. Build zones and safe manual/automatic irrigation control.
6. Add irrigation history and CSV export.
7. Add the alert rules engine and notification preferences.
8. Build plant analysis from rule-based insights, then consider predictive models after historical data exists.
9. Test device failure, stale data, low tank, rain, abnormal flow, and Wi-Fi loss scenarios.
10. Deploy with HTTPS, a managed PostgreSQL database, an MQTT broker, and application monitoring.

Verification checklist

- backend npm run typecheck passes
- backend npm run build passes
- frontend npm run typecheck passes
- frontend npm run build passes
- GET /api/v1/health returns status ok
- POST telemetry accepts valid readings and updates the dashboard summary
- Invalid telemetry returns HTTP 400
- MQTT remains disabled until explicitly configured
- PostgreSQL fallback keeps local development usable without a database
- Frontend connects through REST and Socket.IO

Troubleshooting

Port already in use

The backend uses port 4000 and the frontend uses port 5173 by default. Stop the existing process or set PORT / Vite server configuration before starting another instance.

Dashboard shows offline

Start the backend, confirm the API URL, and submit telemetry. The in-memory store marks a device offline when its latest reading is more than five minutes old.

PostgreSQL is unavailable

Leave DATABASE_URL empty for local UI work. The backend logs the fallback and continues with seeded in-memory telemetry.

MQTT is unavailable

Leave MQTT_ENABLED=false and use the HTTP telemetry endpoint while the broker or ESP32 is not ready.

Source and scope note

The functional scope and sample data in this guide are consolidated from the supplied AgroSense structure document and the implementation currently present in this repository. Product screenshots are treated as visual direction; safety behavior, authentication policy, hardware wiring, notification channels, and plant-analysis formulas still require product decisions before production release.